İSTANBUL DATA SCIENCE ACADEMY

Copilot, MCP ve Token Ekonomisi

Çalışma kitabı

INTELLIJ IDEA / JETBRAINS SÜRÜMÜ

Bağlam bir bütçedir. Bu kitap günün akışını izler: her bölümde önce **konunun özeti**, hemen ardından **o konunun labı**. Bütün lab adımları **JetBrains akışına** göre yazıldı.

EĞITMEN

Zekeriya Beşiroğlu

Zekeriya.besiroglu@istdsa.com

Bir günlük atölye
2026

İçindekiler

00 Nasıl kullanılır ve günün akışı 3
Kitabın yapısı, saat saat program

01 Bölüm 1 — Ne gönderiyorsunuz 4
Talimat dosyaları: hangi kural hangi dosyaya, ve her satırın bedeli

02 Bölüm 2 — Neyi açıyorsunuz 9
MCP: kurum içi kaynağı araç olarak açmak ve sınırlarını çizmek

03 Bölüm 3 — Ne ödüyorsunuz 15
Token ekonomisi: sayaç, krediler, havuzlanma ve üç eşik

04 Bölüm 4 — Optimizasyon 19
Yedi kaldıraç, hangisi kural hangisi alışkanlık

05 Referans kartı · IntelliJ 23
Günün tamamı tek sayfada, JetBrains'e göre

06 IntelliJ'de neyin farklı olduğu 24
VS Code ile birebir karşılaştırma ve üç sonuç

07 Yarın uygulayabileceğiniz üç şey 25
İzin gerektirmeyen üç değişiklik

Bu kitap nasıl kullanılır

Konu özetleri anlatım sırasında takip edilir, sonrasında tekrar okunur. **Lab adımlarında** her adımda üç şey var: ne yaptığınız, komut ve **görmeniz gereken çıktı**. Gördüğünüz çıktı yazandan farklıysa devam etmeyin, o adımda kalın.

Kendinize sorun bölümlerindeki cevaplar kapanış atölyesinde kullanılacak — boş bırakmayın.

Kurulum bu kitapta yok: eğitimden önce reponun içindeki sıfırdan yapma kılavuzuyla (VS Code veya IntelliJ) ortamınızı hazırlayın.

BAŞLANGIÇ

Nasıl kullanılır ve günün akışı

Kitabın yapısı ve saat saat program

Copilot, MCP ve Token Ekonomisi · bir günlük atölye

> Bağlam bir bütçedir. Ne gönderdiğiniz, neyi açtığınız, ne ödediğiniz.

Bu kitap günün akışını izler: her bölümde önce konunun özeti, hemen ardından o konunun labı. Bütün lab adımları **IntelliJ IDEA ve JetBrains IDE'leri** için yazıldı — ekran yolları, komutlar ve beklenen çıktılar JetBrains'e göre.

ÖNEMLI

VS Code kullanıyorsanız bu kitap yanlış kitap: `CALISMA-KITABI.md` sizin sürümünüz. İki IDE'de üç şey gerçekten farklı çalışıyor ve adımlar o farklara göre ayrışıyor.

Nasıl kullanılır

Konu özetleri	Anlatım sırasında takip edin, sonrasında tekrar okuyun
Lab adımları	Komutları kopyalayıp çalıştırın, beklenen çıktıyla karşılaştırın
Kendinize sorun	Her bölüm sonunda; cevaplar kapanış atölyesinde işinize yarayacak

Her lab adımında üç şey var: ne yaptığınız, komut veya ekran yolu ve **görmeniz gereken çıktı**. Gördüğünüz çıktı yazandan farklıysa devam etmeyin.

Kurulum bu kitapta yok. Eğitimden önce `labs/intellij/SIFIRDAN-YAPMA-KILAVUZU.md` ile ortamınızı hazırlayın — H1'den H12'ye kadar on iki adım.

DIKKAT

H12 adımını atlamayın. Orada iki tam yol alıp saklıyorsunuz. Lab 2'de ayar arayüzüne bunları elle yazacaksınız: JetBrains'te `${workspaceFolder}` gibi bir değişken yok, yolu kısaltamazsınız.

Günün akışı

SAAT	BÖLÜM	SÜRE
09:30–11:00	`1 · Ne gönderiyorsunuz`	Özet 45 dk + Lab 45 dk
11:15–12:45	`2 · Neyi açıyorsunuz`	Özet 30 dk + Lab 60 dk
13:45–15:00	`3 · Ne ödüyorsunuz`	Özet 30 dk + Lab 45 dk
15:15–16:30	`4 · Optimizasyon`	Özet 15 dk + Lab 60 dk
16:30–17:00	Kapanış: politika taslağı	Grup çalışması

BÖLÜM 1

Ne gönderiyorsunuz

Talimat dosyaları: hangi kural hangi dosyaya, ve her satırın bedeli

Konu özeti

Copilot'un davranışını repodaki dosyalarla yönetebiliyorsunuz. Dört dosya türü var — ama **JetBrains bunların ikisini hiç okumuyor.**

DOSYA	VS CODE	INTELLIJ
`copilot-instructions.md`	Her istekte	**Her istekte**
`*.prompt.md`	`/komut` yazınca	**`/komut` yazınca**
`*.instructions.md` + `applyTo`	`applyTo` eşleşince	**Okunmuyor**
`*.chatmode.md`	Mod seçilince	**Okunmuyor**

Buna bir de JetBrains'e özel bir dosya ekleniyor:

UYARI

`global-copilot-instructions.md` makinenizde duruyor ve **tüm projelerinize** uygulanıyor. Her isteğe giriyor, repoda görünmüyor, kimse incelemiyor. Kurumsal ortamda bu dosyanın varlığını bilmek gerekir.

Her satırın bir bedeli var

`copilot-instructions.md`'ye yazdığınız her satır, ekipteki herkesin sorduğu her soruya ekleniyor. Somut hesap:

120 kişi × günde 4 istek × 21 gün × 1.000 fazla token	**10,1 milyon token/ay**
Hafif modelde	$2/ay · $24/yıl
Dengeli modelde	$20/ay · $242/yıl
Güçlü modelde	$50/ay · $605/yıl

Rakam, görmezden gelinecek kadar küçük ve utandıracak kadar büyük. Bu dosyaların yıllarca kabararak hayatta kalmasının sebebi tam olarak bu.

Satır satır denetim

Her satır için tek bir soru: *ekibimin sorduğu soruların yüzde kaçında bu satır gerçekten gerekli?*

CEVAP	SATIR NEREYE AIT
%80 ve üstü	Burada kalır
%20 – %80	**Prompt dosyasına** — JetBrains'te `applyTo` yok
%20 altı	Prompt dosyasına, istendiğinde çağrılır

Hedef: her istekte yüklenen dosya 500 token altında.

Kapsamlama yerine çağrı düzeyi

VS Code'da orta satır `instructions/` klasörüne gidiyordu: `applyTo` globu sayesinde bir Java kuralı, siz Python yazarken hiç yüklenmiyordu — otomatik olarak.

JetBrains bunu desteklemiyor. **Kapsamlama fikri hâlâ doğru** — ilgisiz bağlam hem para yakıyor hem modeli şaşırtıyor — ama otomatik aracı yok. Siz çağrı düzeyinde yapıyorsunuz: detayı prompt dosyasına koyup yalnızca gerektiğinde çağırmak aynı işi görüyor, bir tık maliyetiyle.

> Bu yüzden JetBrains'te prompt dosyaları VS Code'dakinden **daha önemli.** Her istekte yüklenen dosyadan çıkardığınız her şeyin gideceği tek yer orası.

Bu dosya kod mu, politika mı?

Dosya kod gibi davranıyor — repoda duruyor, PR'la değişiyor. Ama politika gibi sonuç doğuruyor: bir kişinin eklediği satır, herkesin her sorusunun maliyetine giriyor.

Çoğu kurumda üç sorunun cevabı verilmemiş: kim değiştirebilir, hangi incelemeden geçer, ne zaman yeniden okunur.

Lab 1 · 45 dakika

Hedef: Copilot'un davranışını dosyayla yönetmek ve JetBrains'in bu dosyalardan hangisini gerçekten okuduğunu görmek.

1.1 Önce ölçün

Sohbet panelini açın (sağ kenar çubuğundaki Copilot ikonu), **Agent** modunda olduğunuzu doğrulayın ve hiçbir dosyaya bakmadan sorun:

```
Bu projede para tutarları için hangi tipi kullanmalıyım?
```

Görmeniz gereken: genel bir cevap — *"BigDecimal genelde iyi bir seçim ama duruma göre değişir"* gibi. Cevabı bir yere yazın.

> Talimat dosyasını önce eklerseniz farkı hiç göremezsiniz. **Önce ölç, sonra değiştir** – bugün yapacağınız her ölçümde bu sıra geçerli.

1.2 Talimat dosyasını okuyun, tekrar sorun

Proje ağacından `.github/copilot-instructions.md` dosyasını açıp okuyun. Altı invariant var.

Sonra **yeni bir sohbet açıp** (sohbet panelinin üstündeki **+** düğmesi) birebir aynı soruyu sorun.

Görmeniz gereken: kesin bir cevap — `BigDecimal`, asla `double`.

UYARI

Yeni sohbet şart. Aynı sohbette sorarsanız ilk cevap hâlâ bağlamda durur ve ikincisi kirlenir. Lab 3'te aynı hata bütün ölçümünüzü bozar.

1.3 Dosyaları tartın — ve neyin okunmadığını görün

IntelliJ'in alt panelindeki **Terminal** sekmesini açın, sanal ortamın etkin olduğunu doğrulayın (satır başında `(.venv)` görünmeli) ve çalıştırın:

```
python -c "
import sys; sys.path.insert(0,'.')
from cost.calculator import estimate_tokens
from pathlib import Path
t=0
for f in sorted(Path('.github').rglob('*.md')):
    n=estimate_tokens(f.read_text()); t+=n
    print(f'{n:>6} token  {f}')
print(f'{t:>6} token  TOPLAM')
"
```

Görmeniz gereken:

```
   279 token  .github/chatmodes/analyst.chatmode.md
   292 token  .github/copilot-instructions.md
   130 token  .github/instructions/data-jobs.instructions.md
   310 token  .github/instructions/java-service.instructions.md
   292 token  .github/prompts/analist.prompt.md
   188 token  .github/prompts/release-check.prompt.md
   221 token  .github/prompts/service-review.prompt.md
  1712 token  TOPLAM
```

Şimdi asıl mesele. **Bu 1712 token'ın hepsi IntelliJ'de okunmuyor:**

DOSYA	TOKEN	INTELLIJ'DE
`copilot-instructions.md`	292	**Her istekte yüklenir**
`prompts/*.prompt.md`	701	Yalnızca `/komut` ile çağrılınca
`instructions/*.md`	440	**Okunmuyor**
`chatmodes/*.md`	279	**Okunmuyor**

Her isteğe eklenen tek şey **292 token**. 719 token repoda duruyor ama hiçbir şey yapmıyor.

> NOT
>
> Okunmayan dosyalar maliyet de üretmiyor — ama fayda da sağlamıyor. Ekibinizde hem VS Code hem IntelliJ kullanılıyorsa **aynı repo iki kişide farklı davranıyor.** Bu, Lab 1'in tartışma sorusu.

Şimdi kendi kurumunuzun dosyasını aynı komutla ölçün. Çoğu kurumda rakam üç ila on kat büyük çıkıyor.

1.4 Her satırı denetleyin

`copilot-instructions.md` dosyasını satır satır gezin ve her satır için yukarıdaki %80 testini uygulayın.

Sonra `.github/instructions/java-service.instructions.md` dosyasını açın — 310 token, ve **IntelliJ bunu okumuyor.** Şu soruyu bir cümleyle yanıtlayın:

> Bu içerik JetBrains'te nereye ait? Hepsi her isteğe mi girmeli, yoksa bir kısmı istendiğinde mi çağrılmalı?

Kararınızı yazın — Lab 4'te uygulayacaksınız.

	KARARINIZ
Her isteğe girmeli	
Prompt dosyasına taşınmalı	
Tamamen silinmeli	

1.5 Prompt dosyalarını deneyin

Bir `.java` dosyası açıkken sohbete yazın:

```
/service-review
```

Görmeniz gereken: eğik çizgiyi yazdığınızda açılan listede `service-review`, `release-check` ve `analist`.

LISTE ÇIKMIYORSA

- `.github/prompts/` klasörünün var olduğunu doğrulayın
- IntelliJ'in **repo kökünü** açtığını doğrulayın — alt klasörü açtıysanız Copilot dosyaları bulamaz (hazırlık H7)
- IDE'yi tamamen kapatıp açın

Kendi promptunuzu yazın

Haftada en az bir kez tekrarladığınız bir işi seçin. `.github/prompts/` altına yeni dosya oluşturun:

```
---
description: Değişiklikler için PR açıklaması yazar
mode: agent
---

# PR açıklaması

1. Değişen dosyaları incele.
2. Şu başlıklarla bir açıklama yaz:
   - Ne değişti (madde madde)
   - Neden
   - Nasıl test edildi
3. Kod değiştirme. Sadece metni ver.
```

Üç kural: **çok adımlı**, **açık çıktı biçimi**, **sonunda bir duruş** ("önce raporu ver, onay bekle").

Dosyayı kaydedin, sohbette `/` yazıp listede göründüğünü doğrulayın.

1.6 Analist davranışını deneyin

VS Code'da bu bir **chat mode**'du. JetBrains'te chat mode yok; davranışı prompt dosyasıyla taşıyoruz.

`.github/prompts/analist.prompt.md` dosyasını açıp okuyun. Sonra sohbete:

```
/analist
```

Ardından sırayla sorun:

```
Sürüm takvimi kuralları nerede tanımlı ve ne diyor?
```

```
Bana bir servis sınıfı yaz.
```

Görmeniz gereken: ikincisini **reddetmeli**.

İki farkı not edin

	CHAT MODE (VS CODE)	PROMPT DOSYASI (JETBRAINS)
Ne kadar sürer	Mod seçili kaldığı sürece	O sohbet boyunca
Araç erişimi	`tools` ile kısıtlanır	**Kısıtlanamaz**
Nasıl açılır	Mod menüsünden	`/analist` yazarak

NOT

İkinci satır önemli: chat mode'da araç kısıtlaması bağlamı da küçültüyordu — az araç, az şema, ucuz cevap. Burada **davranışsal sınır kalıyor, maliyet kazancı kalmıyor.**

Lab 1 çıktıları

- [ ] 1.1 ve 1.2'deki iki cevap yazılı, farkı görüldü
- [ ] 1712 token'ın hangi kısmının okunduğu görüldü (292)
- [ ] Kapsamlama olmadığında ne yapılacağına dair karar yazıldı
- [ ] Kendi `.prompt.md` dosyanız `/komut` ile çalıştı
- [ ] `/analist` denendi, reddedişi görüldü

Kendinize sorun

1. Ekibinizde hem VS Code hem IntelliJ kullanılıyorsa, **aynı repo iki kişide farklı davranıyor.** Bunu nasıl yönetirsiniz — tek IDE'ye mi standartlaşmak, yoksa yalnızca her ikisinin de okuduğu dosyaya mı yazmak?
2. Kurumunuzda bu dosyaları kim yazar ve kim onaylar? Bir geliştirici bir satır eklediğinde maliyet herkese yayılıyor. **Bu dosya kod mudur, politika mıdır?**
3. `global-copilot-instructions.md` makinede duruyor ve hiç incelenmiyor. Bunun kurumunuzda bir karşılığı olmalı mı?

	NOTUNUZ
Dosyamın token sayısı	
IntelliJ'in okuduğu kısım	
Taşınacak satır sayısı	

BÖLÜM 2

Neyi açıyorsunuz

MCP: kurum içi kaynağı araç olarak açmak ve sınırlarını çizmek

Konu özeti

MCP (Model Context Protocol), modelin dış dünyayla konuşması için standart bir arayüz. Üç şey sunuyor: **tools** (çağrılabilir fonksiyonlar), **resources** (okunabilir içerik), **prompts** (hazır şablonlar).

ÖNEMLI

Copilot yalnızca tools kullanıyor. Resources ve prompts desteklenmiyor — ve size bunu kimse söylemez. Hata almazsınız, sadece beklediğiniz şey olmaz.

Bankada dört kritik nokta

Bir — yerel çalışmak, içeride kalmak değildir.

Zinciri üç adımda görün:

1. Aracın döndürdüğü her şey bağlama giriyor — girmek zorunda, model onun üzerinde akıl yürütecek
2. Bağlam model sağlayıcısına gidiyor — bu turda ve sonraki her turda
3. Dolayısıyla araç bir **veri yolu**

Bir MCP aracı, kimse buna karar vermeden sınırınızın dışına veri taşıyan bir kanala dönüşebilir. Bu kurumsal bir karardır, geliştirici kararı değildir.

İki — onay akışı denetim izidir.

SEÇENEK	NE ELDE EDERSINIZ
Her çağrıda onay	Her kullanım, bir insanın belirli bir anda verdiği karar
Otomatik onay	Kararların kaydı, faaliyetin kaydına dönüşür
Orta yol	Salt-okunur araçlar otomatik; yazanlar asla

Üç — onaylı liste, serbest kurulum değil.

Bir onayın yanıtlaması gereken üç soru: **ne döndürüyor** (hangi alanlar, kişisel veri var mı), **kim sahibi** (araç yanlış şey döndürdüğünde cevap verecek ekip), **kim bağlayabilir**.

Dört — JetBrains'te yapılandırma görünmez.

	VS CODE	INTELLIJ
Yapılandırma nerede	Repoda, `.vscode/mcp.json`	**IDE ayarlarında**
Ekiple paylaşılır mı	Evet, commit edilir	**Hayır**
PR'da incelenir mi	Evet	**Hayır**

> Üçüncü satır bankada konuşulmaya değer: repoda duran bir yapılandırma **incelenebilir bir karardır.** IDE ayarında duran, kimsenin görmediği bir karardır.

Araç bütçesi

Bağlı olan her sunucu, araç şemalarını **her isteğe** ekliyor — o sunucuyu o soruda kullanıp kullanmamanız fark etmiyor.

BAĞLI SUNUCU	HER ISTEĞE EKLENEN ŞEMA (8 ARAÇ/SUNUCU)
1 sunucu	8
3 sunucu	24
10 sunucu	80

Copilot'un tavanı istek başına 128 araç, ama kalite bozulması çok daha önce başlıyor — model bunlar arasından *seçmek* zorunda.

> **Kural:** bugün kullanmadığınız sunucuyu kapatın.

İyi araç tasarımı

Model araç seçimini **açıklamaları okuyarak** yapıyor:

KÖTÜ	İYI
"Servis sahibi bilgisini döndürür."	"Bir servisten sorumlu ekip gerektiğinde kullan — kayıt açmadan veya olayı yükseltmeden önce. Kısmi isim varsa önce `api_search` çağır."

Üç kural: **faydalı biçimde başarısız olun** (mevcut seçenekleri döndürün), **gerekçeyi de döndürün** (modelin okuduğu gerekçe, uyduramayacağı gerekçedir), **açıklamaları kısa tutun** (her kelime her istekte gidiyor).

Lab 2 · 60 dakika

Hedef: kurum içi bir kaynağı Copilot'a araç olarak açmak ve sınırlarını görmek.

ÖNEMLI

VS Code'dan en büyük fark bu labda. Orada repoda duran bir `.vscode/mcp.json` dosyası vardı ve ekiple paylaşılıyordu. JetBrains'te yapılandırma IDE ayarlarında duruyor ve elle yazılıyor.

2.1 Sunucuyu elle bir kez çalıştırın

IDE'ye dokunmadan önce, sunucunun çalıştığını kanıtlayın. IntelliJ'in **Terminal** sekmesinde:

```
"$(pwd)/.venv/bin/python" "$(pwd)/mcp_server/corp_server.py"
```

Windows'ta:

```
"%CD%\.venv\Scripts\python.exe" "%CD%\mcp_server\corp_server.py"
```

Görmeniz gereken: hiçbir şey. Terminal donar ve bekler — **bu doğru**, sunucu stdin dinliyor. `Ctrl+C` ile çıkın.

SORUN ÇIKARSA

Traceback dökerse — hata sunucu kodunda ya da kurulumda. Mesajı okuyun; IDE'nin sizden saklayacağı şeyi burada görüyorsunuz.

Hemen komut istemine dönerse — komutu gerçek bir terminalde çalıştırdığınızdan emin olun. IDE'nin **Run** düğmesiyle çalıştırırsanız stdin kapalı olur ve sunucu sessizce çıkar.

`no such file or directory` — ya repo kökünde değilsiniz (`pwd`) ya da `.venv` kurulmamış (`ls -d .venv`).

2.2 MCP'yi ayar arayüzünden tanımlayın

Dört adım:

1. Sağ alt köşedeki **Copilot ikonuna** tıklayın
2. **Edit Settings** seçin
3. **Model Context Protocol** bölümüne gidin
4. **Configure** düğmesine basın

Açılan `mcp.json` düzenleyicisine şunu yazın — **yolları hazırlık H12'de aldığınız iki değerle değiştirerek**:

```
{
  "servers": {
    "corp-internal": {
      "type": "stdio",
      "command": "/BURAYA/KOMUT/YOLU",
      "args": ["/BURAYA/ARGUMAN/YOLU"]
    }
  }
}
```

`Cmd+S` / `Ctrl+S` ile kaydedin veya dosyayı kapatın.

DIKKAT

Düz `python` yazmayın. PATH'te ilk gelen yorumlayıcı çalışır — genelde sistem Python'ı veya Anaconda base ortamı. Orada `mcp` paketi olmadığı için süreç anında ölür ve **sebebi görünmez**: ne hata mesajı, ne log.

İki yolun da içinde `.venv` geçtiğini gözünüzle kontrol edin.

2.3 Sunucuyu doğrulayın

Sohbet panelini açın, **Agent** modunda olduğunuzu doğrulayın ve yazın:

```
standards_list aracını çalıştır
```

Görmeniz gereken: araç çağrısı için bir **izin istemi**. İzin verdiğinizde üç standardın listesi dönecek: `java-service` , `data-access` , `ai-assistant-usage` .

SORUN GIDERME

BELIRTI	SEBEP VE ÇÖZÜM
Araç bilinmiyor diyor	Agent modunda değilsiniz
Sunucu başlamıyor	`command` yanlış yorumlayıcı — 2.2'ye dönün
Hiçbir şey olmuyor	IDE'yi tamamen kapatıp açın
Ayarlarda MCP bölümü yok	Copilot eklentisi eski; güncelleyin
Kurumsal hesapta kapalı	Yönetici politikası — eğitmene söyleyin

2.4 Üç soruyu sorun

Her birini **ayrı sohbette** sorun:

```
Hangi dahili API'ler kişisel veri içermiyor?
```

```
Bugün üretime çıkabilir miyiz?
```

```
14 Ağustos 2026'da deploy edebilir miyiz?
```

Görmeniz gereken: üçüncüsü **hayır** demeli ve gerekçesini de vermeli — *"Month-end reconciliation window"*.

> Araç sadece `false` döndürmüyor, nedenini de döndürüyor. Model gerekçeyi uyduramıyor, alıntılamak zorunda.
>
> Her çağrıdan önce gelen izin istemini okuyun. O istem bu labın ana konusu: her araç çağrısı, bir insanın belirli bir anda verdiği bir karar.

2.5 Geliştirici kolu — araç yazın

> Analistseniz `2.6 Analist kolu` adımına geçin.

`mcp_server/corp_server.py` dosyasında `tool_cost` fonksiyonunun **üstüne** ekleyin:

```
@mcp.tool()
def find_service_owner(service_name: str) → str:
    """Bir servisten sorumlu ekibi döndürür.

    Kullanıcının bir servis hakkında kime başvuracağını bilmesi gerektiğinde
    kullan — kayıt açmadan veya olayı yükseltmeden önce. Elinde yalnızca
    kısmi bir isim varsa önce api_search çağır.

    Args:
        service_name: API kataloğundaki anahtar veya ad.
    """
    # gövdeyi siz yazın
```

Üç tasarım kuralı:

- Docstring **"ne zaman çağrılacağını"** söylesin, "ne yaptığını" değil
- Bulunamayınca **mevcut seçenekleri döndürün**
- **Kişisel veri döndürmeyin**

`tests/test_mcp_tools.py` dosyasının import satırına `find_service_owner` ekleyin, sonra testleri yazın:

```
def test_find_service_owner_returns_the_team():
    result = call(find_service_owner, service_name="limit-engine")
    assert result["owner_team"] == "Credit Technology"


def test_unknown_service_lists_alternatives():
    result = call(find_service_owner, service_name="nope")
    assert "error" in result
    assert "limit-engine" in result["available"]
```

```
python -m pytest tests -q
```

Görmeniz gereken: `30 passed`

> `NAMEERROR` ALIYORSANIZ
>
> Import satırını atladınız. `tests/test_mcp_tools.py` başındaki listeye `find_service_owner` ekleyin.

Sunucuyu yeniden yükleyin

> UYARI
>
> **JetBrains'te `MCP: Restart Server` komutu yok.** Copilot ayarlarından MCP yapılandırmasını açıp yeniden kaydedin, ya da IDE'yi tamamen yeniden başlatın. Kodu değiştirip sohbete dönmek yetmez — eski süreç çalışmaya devam eder.

Sonra sohbete: `Limit motorunun sahibi kim?`

2.6 Analist kolu — araçları değerlendirin

Araç yazmıyorsunuz. Daha zor olan soruyu yanıtlıyorsunuz: **hangi araç olmamalı.**

`mcp_server/corp_server.py` dosyasını açın, beş docstring'i okuyun ve beş soruya **yazılı** cevap verin:

1. Hangi açıklama modelin **yanlış aracı** seçmesine yol açabilir? Yeniden yazın.
2. `api_search` aracının `pii_free_only` parametresi olmasaydı ne bozulurdu?
3. Bu sunucuya **eklenmemesi** gereken bir araç söyleyin ve nedenini savunun. Somut olun: hangi veriyi döndüreceğini ve neyin ters gideceğini adıyla yazın.
4. `can_deploy` verdiğin yanında gerekçe de döndürüyor. Sadece `true` / `false` dönseydi ne kaybederdik?
5. **JetBrains'e özel:** yapılandırma repoda değil IDE ayarlarında duruyor. Bir denetçi *"bu ekip hangi MCP sunucularını bağlamış"* diye sorsa, cevabı nereden bulursunuz?

> Bu kolay kol değil. Bankada asıl kritik soru bir aracın *yazılmaması* gerektiği, ve bu soru Python bilene değil iş sonucunu bilene ait. Değerlendirmede ilk siz sunuyorsunuz.

2.7 Sunucunun maliyetini görün

```
tool_cost aracını çalıştır
```

Her aracın yaklaşık token sayısını ve toplamı göreceksiniz. Bu şemalar, **bu sunucu bağlıyken yapılan her isteğe** gidiyor — sunucuyla hiç ilgisi olmayan istekler dahil.

Hesabı yapın: sekizer araçlı on sunucu, sorduğunuz **her sorunun** başına seksen araç şeması ekler.

Lab 2 çıktıları

- [ ] Sunucu elle çalıştırıldı ve dondu
- [ ] Ayar arayüzünden **tam yollarla** tanımlandı
- [ ] İzin istemi görüldü, üç standart döndü
- [ ] Üç soru soruldu, üçüncüsü gerekçeli hayır döndü
- [ ] Geliştirici: `find_service_owner` yazıldı, `30 passed`, canlı çalışıyor
- [ ] Analist: beş soruya yazılı cevap
- [ ] `tool_cost` çalıştırıldı, hesap yapıldı

Kendinize sorun

1. Bir ekip müşteri sorgulama aracı yazmak isterse **hangi onaylardan geçmeli** ve kararı kim verir: geliştirici mi, mimari mi, uyum mu?
2. Yapılandırma IDE ayarlarında durduğu için ekip genelinde görünür değil. Bir denetçi *"geçen ay hangi veri bir araç çağrısı üzerinden sınırın dışına çıktı"* diye sorsa, kurumunuzda **kim cevap verebilir**?
3. VS Code'da `mcp.json` PR'da incelenebiliyordu. JetBrains'te bu incelemeyi neyin yerine koyarsınız — merkezi bir onaylı liste mi, periyodik denetim mi?

	NOTUNUZ
Kurumumuzda MCP onayını kim verir	
Denetim sorusunun cevabı var mı	
İnceleme neyin yerine konacak	

BÖLÜM 3

Ne ödüyorsunuz

Token ekonomisi: sayaç, krediler, havuzlanma ve üç eşik

NOT

Bu bölümün tamamı IDE'den bağımsız. Token ekonomisi VS Code'da da IntelliJ'de de birebir aynı çalışıyor — aynı modeller, aynı fiyatlar, aynı eşikler. Komutları IntelliJ'in Terminal sekmesinde çalıştırıyorsunuz.

Konu özeti

Üç şey sayılıyor: **girdi** (gönderdiğiniz her şey, her turda), **çıktı** (modelin yazdığı, girdiden birkaç kat pahalı), **önbellekli girdi** (değişmeyen bağlam, onda bir fiyata).

Her şey tek bir dönüşüme iniyor: **1 AI kredisi = 1 sent.**

ÖRNEK BIR ISTEK	
Girdi — 100.000 token × $2,00/1M	$0,20
Çıktı — 3.000 token × $12,00/1M	$0,04
Toplam	**$0,24 = 24 kredi**

Tek istek önemsiz. 120 kişi × 4 istek × 21 gün değil. Birim küçük, çarpan kadronuz.

Havuzlanma

Koltuk başına dahil kredi Business'ta $19, Enterprise'da $39. Ama havuz koltuk başına değil — **fatura varlığı düzeyinde toplanıyor**. 120 koltuklu bir kurumda aylık havuz $2.280.

Pratikte: bir ağır kullanan ekip, birkaç hafif ekibin başlığını tüketebilir — ve araçlarda hiçbir şey bunu göstermiyor, fatura gelene kadar.

Üç eşik

EŞIK	KAT	NEREDEN GELIYOR
Model seçimi	25×	Hafif $0,0236 ↔ güçlü $0,5900 (aynı istek)
Önbellek	10×	Değişmeyen bağlam onda bir fiyata
Uzun bağlam	2×	272K aşılınca birim fiyat ikiye katlanıyor

UYARI

Önbellek bedava değil. Anthropic modellerinde önbelleğe **yazmak** düz girdiden pahalı (Sonnet 5: girdi $2,00, önbellekli $0,20, yazma $2,50). Önbellek bir yatırım — tekrar kullanımda geri dönüyor, kısa ömürlü bağlamda zarar bile ettirebiliyor.

Kaldıraçlar çarpılıyor

DEĞIŞEN	MALIYET	KAZANÇ
Hiçbiri	$0,5110	—
Yalnız model hafifledi	$0,0204	25×
Yalnız bağlam küçüldü	$0,0570	9×
İkisi birden	**$0,0023**	**224×**

Kaba hesap 25 × 9 = 225, ölçülen 224×. **Kaldıraçlar toplanmıyor, çarpılıyor.**

Lab 3 · 45 dakika

3.1 Gerçek rakamınızı bulun

Tarayıcıda: **GitHub → Settings → Billing → Usage**

	SIZIN RAKAMINIZ
Bu ay harcanan AI Credits	
Karışımdaki modeller	
Dahil kredinin yüzde kaçı	

DIKKAT

Bu adımı atlamayın. Sonraki her şey bu rakama dayanıyor. *Tahminle bütçe kurmak, ölçmemenin kibarcasıdır.*

3.2 Senaryo scriptini çalıştırın ve kalibre edin

IntelliJ Terminal'de:

```
python cost/scenarios.py
```

Sonra `cost/scenarios.py` dosyasını proje ağacından açıp üç sabiti kendi ekibinize göre değiştirin:

```
SEATS = 120          # kendi ekip büyüklüğünüz
WORKDAYS = 21
ALLOWANCE = 19.0     # Business 19, Enterprise 39
```

Tekrar çalıştırıp **3.1'deki gerçek rakamla karşılaştırın.** Fark büyükse hatalı olan hesaplayıcı değil, sizin kullanım varsayımlarınız.

3.3 Üç eşiği doğrulayın

```
python -c "
import sys; sys.path.insert(0,'.')
from cost.calculator import Interaction
for m in ('gpt-5.6-luna','gpt-5.6-terra','gpt-5.6-sol'):
    i = Interaction(m, input_tokens=100_000, output_tokens=3_000)
    print(f'{m:>16}  \${i.cost_usd():.4f}  {i.credits():>6.1f} kredi')
"
```

Görmeniz gereken:

```
    gpt-5.6-luna  $0.0236     2.4 kredi
   gpt-5.6-terra  $0.2360    23.6 kredi
     gpt-5.6-sol  $0.5900    59.0 kredi
```

```
python -c "
import sys; sys.path.insert(0,'.')
from cost.calculator import Interaction
tam = Interaction('gpt-5.6-terra', input_tokens=100_000)
onb = Interaction('gpt-5.6-terra', cached_input_tokens=100_000)
print(f'oran {tam.cost_usd()/onb.cost_usd():.1f}x')
"
```

Görmeniz gereken: `oran 10.0x`

```
python -c "
import sys; sys.path.insert(0,'.')
from cost.calculator import Interaction
for n in (270_000, 280_000):
    i = Interaction('gpt-5.6-terra', input_tokens=n, output_tokens=3_000)
    print(f'{n:>9,} token → \${i.cost_usd():.4f}')
"
```

Görmeniz gereken: `$0.5760` ve `$1.1740` — %3,7 daha çok token, %104 daha çok para.

3.4 Kendinizi ölçün (A/B koşusu)

Bir görev seçin. İki kez yapın. Model seçimini sohbet panelinin altındaki model menüsünden değiştiriyorsunuz.

- **A:** beş dosya bağlamda, güçlü model, uzun sohbetin ortasında
- **B:** yeni sohbet, tek dosya, hafif model, net istek

	A KOŞUSU	B KOŞUSU
Kullanılan model		
Bağlamdaki dosya sayısı		
Harcanan kredi		
Cevap kabul edilebilir mi?		

ÖNEMLI

Dördüncü satır boşsa ölçüm geçersizdir. B'nin cevabını gerçekten kullanır mıydınız, yoksa A'yı görmüş olduğunuz için mi kabul edilebilir göründü?

Lab 3 çıktıları

- [ ] Billing ekranından gerçek rakam alındı
- [ ] `scenarios.py` kendi ekibinize kalibre edildi
- [ ] Üç eşik komutla doğrulandı: 25×, 10×, 2×
- [ ] A/B koşusu yapıldı, dört satır da dolduruldu

Kendinize sorun

1. Havuzlanmış kredide bir ekibin aşımı herkesin sorunu. **Bütçe kimin olmalı** — ekip lideri mi, platform ekibi mi?
2. Havuz aşıma girdiğinde **ilk hangi veri** istenmeli?

	NOTUNUZ
Bizim ekibin aylık tahmini	
Kullanım raporuna kim erişebiliyor	

BÖLÜM 4

Optimizasyon

Yedi kaldıraç, hangisi kural hangisi alışkanlık

Konu özeti

Yedi kaldıraç var ve sıra önemli: listeyi yukarıdan aşağı uygulayın, çünkü ilk üçü tasarrufun çoğunu veriyor.

#	KALDIRAÇ	NE DEMEK	JETBRAINS'TE
1	Doğru yüzey	Tamamlama ölçülmüyor; o işi sohbete taşımak bedavayı ücretliye çevirir	Aynı
2	Model yönlendirme	Aynı istekte 25 kata kadar fark	Aynı
3	Bağlam disiplini	Eklenen her dosya her turda yeniden gidiyor	Aynı
4	Önbellek sıralaması	Sabit kısmı sabit tutun	Aynı
5	Araç bütçesi	Kullanmadığınız sunucuyu kapatın	**Ayar arayüzünden**
6	Yeni sohbet refleksi	Konu değiştiyse baştan başlayın	Aynı
7	Yeniden deneme	İyi yazılmış tek istek, üç kötüden ucuz	Aynı

İlk üçünün tetiği

Alışkanlığın somut bir tetiği olmalı, yoksa yerleşmiyor:

KALDIRAÇ	TETİK	ONUN YERİNE
Doğru yüzey	Kodu sohbete yapıştırmak üzeresiniz	Satır içi yapın
Model yönlendirme	Modeli düşünmeden sohbet açıyorsunuz	Hafifle başlayın, yetmezse yükseltin
Bağlam disiplini	Dosya yerine klasör ekliyorsunuz	Sorunun ihtiyacı kadarını ekleyin

Kural mı, alışkanlık mı?

Yanlış mekanizma ikisini birden harcar.

KALDIRAÇ	MEKANIZMA
2 · Model yönlendirme	Varsayılan ayar
4 · Önbellek sıralaması	Politika
5 · Araç bütçesi	Politika + onaylı liste
1, 3 · Yüzey ve bağlam	Eğitim + örnek
6, 7 · Yeni sohbet, yeniden deneme	Yalnızca alışkanlık

"Yeni sohbet aç" diye bir politika yazarsanız iki şey olur: kural işlemez ve kuralı yazan kişi itibar kaybeder. **Denetlenemeyen bir kural, kural değil temennidir.**

NOT

JetBrains'te 5. satır daha zor. Araç bütçesi politikası yazabilirsiniz ama uygulanıp uygulanmadığını merkezi olarak göremezsiniz — yapılandırma her geliştiricinin IDE ayarında. Bu, kuralı değil **denetim mekanizmasını** değiştirmenizi gerektiriyor.

Ucuzlatmak hedef değil

İşi yapamayan bir kurulum üreten optimizasyon, optimizasyon değildir.

Başarısızlık biçimi: fatura düşüyor, kabul oranı da düşüyor, insanlar işi elle yapmaya dönüyor — ve kimse raporlamıyor, çünkü herkesin baktığı metrik iyi görünüyor.

Kontrol basit: **kabul edilebilirliği maliyetle aynı tabloya yazın.**

Lab 4 · 60 dakika

4.1 Her dosyayı tartın

```
python -c "
import sys; sys.path.insert(0,'.')
from cost.calculator import estimate_tokens
from pathlib import Path
toplam = 0
for f in sorted(Path('.github').rglob('*.md')):
    t = estimate_tokens(f.read_text()); toplam += t
    print(f'{t:>6} token  {f}')
print(f'{toplam:>6} token  TOPLAM')
"
```

Şimdi **kendi reponuzda** çalıştırın. O rakam bugün üzerinde çalışacağınız şey.

Ayrıca makinenizde `global-copilot-instructions.md` varsa onu da ölçün — her isteğe o da giriyor.

4.2 Always-on dosyayı kesin

Hedef: 500 token altı. Her satıra Lab 1'deki testi uygulayın.

İÇERIK	JETBRAINS'TE NEREYE
Teknolojiye özgü kurallar	`.prompt.md` — `applyTo` yok
Kod inceleme kontrol listesi	`.prompt.md`
Onboarding metni	Wiki — model bunu okumuyor bile

Tasarrufu paraya çevirin:

KESILEN	AYLIK TOKEN	DENGELI MODEL	YILLIK
500 token	5,04M	$10,08	$121
1.000 token	10,08M	$20,16	$242
2.000 token	20,16M	$40,32	$484

DIKKAT

Sahte sonucun işareti: dosyayı küçülttünüz ama hiçbir satırı **taşımadıysanız**, muhtemelen gerçekten gerekli kuralları sildiniz.

4.3 Okunmayan dosyalara karar verin

Bu adım JetBrains'e özel. Lab 1.3'te gördünüz: `instructions/` ve `chatmodes/` klasörleri **719 token** tutuyor ve IntelliJ hiçbirini okumuyor.

Üç seçenek, birini seçip gerekçesini yazın:

SEÇENEK	NE ZAMAN DOĞRU
İçeriği prompt dosyalarına taşı	Kural değerli ve gerektiğinde çağrılabilir
Olduğu gibi bırak	Ekipte VS Code kullananlar da var
Sil	Kural artık geçerli değil veya kimse kullanmıyor

	KARARINIZ VE GEREKÇESI
`instructions/`	
`chatmodes/`	

4.4 Model yönlendirme tablonuzu yazın

İŞ TÜRÜ	MODEL SINIFI	GEREKÇE
Satır içi tamamlama	varsayılan	Hiç ölçülmüyor
Basit soru, kod okuma	hafif	Muhakeme gerekmiyor
Hata ayıklama, refactor	dengeli	Günlük işin dürüst ortası
Mimari, karmaşık agent işi	güçlü	Fiyatını hak ediyor
Kod incelemesi	seçici	Koşu başına pahalı

İyi bir tablo: **model adı değil sınıf** yazar, **iş türüne göre** kurulur, **aşma yolunu** tanımlar.

DIKKAT

Bu tabloyu `copilot-instructions.md` 'ye koymayın. Maliyeti düşürmek için yazdığınız tablo her isteğe eklenerek maliyeti artırır. Wiki'ye koyun.

4.5 Gerçek işte önce/sonra ölçün

	ÖNCE	SONRA
Kullanılan model		
Bağlamdaki dosya sayısı		
Harcanan kredi		
Cevap kabul edilebilir mi?		

KALITE DÜŞTÜYSE NE YAPMALI

Sırayla deneyin: **1)** bağlamı aynı bırakın, yalnızca modeli yükseltin. **2)** model aynı kalsın, yalnızca bir dosya ekleyin.

Bu ikisini **ayrı ayrı** yapın. Aynı anda ikisini birden geri alırsanız hangisinin gerektiğini asla öğrenemezsiniz — ve gereksiz olanı sonsuza kadar ödersiniz.

4.6 Eylem planınızı yazın

	PLANINIZ
Yarın uygulayacağım kural	
Bu çeyrekte yapılacak değişiklik	
Hâlâ ihtiyacım olan veri	

Lab 4 çıktıları

- [ ] Kendi reponuzun `.github/` toplamı ölçüldü
- [ ] Always-on dosya 500 token altına indirildi, satırlar **taşındı**
- [ ] Okunmayan 719 token'a dair karar yazıldı
- [ ] Model yönlendirme tablosu yazıldı (wiki'ye, repoya değil)
- [ ] Önce/sonra ölçüm yapıldı, kabul edilebilirlik satırı doldurulduu

Kendinize sorun

1. Yedi kaldıraçtan hangisi **kural olarak dayatılabilir**, hangisi ancak alışkanlıkla yerleşir?
2. Araç bütçesi JetBrains'te merkezi olarak görülemiyor. Bu kuralı nasıl denetlersiniz?

EK

Referans kartı · IntelliJ

Günün tamamı tek sayfada, JetBrains'e göre

DOSYALAR	
`copilot-instructions.md`	Her istekte · 500 token altı
`*.prompt.md`	`/komut` ile · JetBrains'te asıl araç
`*.instructions.md`	**Okunmuyor**
`*.chatmode.md`	**Okunmuyor**
`global-copilot-instructions.md`	Makinede, tüm projelere, incelenmeden

> **Kural:** her soruda gerekli değilse, her istekte yüklenen dosyaya girmez.

MCP	
Yapılandırma	Copilot ikonu → Edit Settings → Model Context Protocol → Configure
Yol biçimi	**Tam yol** — `${workspaceFolder}` yok
Yeniden başlatma	Ayarı kaydet veya IDE'yi yeniden başlat
Agent modu	Şart — Ask modunda araçlar çalışmaz
Desteklenen	Yalnızca tools
Tavan	İstek başına 128 araç
Uzak sunucu	PAT gerekiyor — OAuth yok

> **Kural:** yerel içeride demek değil. Kullanmadığınızı kapatın.

MALIYET	
Ölçülen	Girdi + çıktı + önbellek
Birim	1 AI kredisi = 1 sent
Dahil kredi	$19 Business / $39 Enterprise · havuzlanır
Ölçülmeyen	Kod tamamlama

> **Kural:** Billing → Usage'dan ölçün, asla tahmin etmeyin.

ÜÇ EŞIK	
Model seçimi	**25×**
Önbellek	**10×** ucuz (yazma maliyeti ayrı)
272Kuzun bağlam	**2×** pahalı

> **Kaldıraçlar çarpılıyor:** 25× ile 9× birlikte **224×** ediyor

EK

IntelliJ'de neyin farklı olduğu

VS Code ile birebir karşılaştırma ve üç sonuç

KONU	VS CODE	INTELLIJ
`copilot-instructions.md`	Var	Var
`applyTo` ile kapsamlama	Var	**Yok**
Chat mode	Var	**Yok** — prompt dosyasıyla taklit
Prompt dosyaları	Var	Var, **daha da önemli**
MCP yapılandırması	Repoda `mcp.json`	**IDE ayarlarında**
Yapılandırma incelenir mi	PR'da evet	**Hayır**
Sunucu yeniden başlatma	`MCP: Restart Server`	Ayarı kaydet veya IDE'yi yeniden başlat
Uzak MCP kimlik doğrulama	OAuth	**PAT**
Token ekonomisi	—	**Tamamen aynı**

Üç sonuç:

1. **Repodaki 1712 token'ın 719'u IntelliJ'de ölü.** Fayda da üretmiyor, maliyet de. Ama ekipte iki IDE varsa aynı repo iki kişide farklı davranıyor.
2. **Prompt dosyaları tek taşıma kabı.** `applyTo` ve chat mode olmadığı için, always-on dosyadan çıkardığınız her şeyin gideceği yer orası.
3. **MCP kararları görünmez.** İnceleme PR'da değil, başka bir mekanizmada olmalı.

EK

Yarın uygulayabileceğiniz üç şey

İzin gerektirmeyen üç değişiklik

1. Satır içi tamamlamanın halledebileceği işi sohbete taşımayı bırakın — izin gerekmiyor.
2. Yeni sohbeti hafif modelle açın; yetmezse yükseltin.
3. Konu değiştiğinde yeni sohbet açın.

NOT

Bu kitaptaki maliyet rakamları modellenmiş kullanım varsayımlarına dayanıyor ve büyüklük mertebesi göstermek içindir. Kendi gerçek rakamınız her zaman **GitHub → Settings → Billing → Usage** ekranındadır.

KÜNYE

Bu kitap hakkında

Kaynaklar, doğrulama ve notlarınız için yer

EĞITIM

Copilot, MCP ve Token Ekonomisi
Bir günlük atölye

KURUM

İstanbul Data Science Academy

EĞITMEN

Zekeriya Beşiroğlu
Zekeriya.besiroglu@istdsa.com

LAB MATERYALI

Katılımcı reposu
MCP sunucusu, maliyet hesaplayıcısı, 28 test

RAKAMLAR HAKKINDA

Bu kitaptaki maliyet senaryoları **modellenmiş kullanım varsayımlarına** dayanıyor; kanıt değil, büyüklük mertebesi göstergesi. Kendi gerçek rakamınız her zaman **GitHub → Settings → Billing → Usage** ekranındadır.

Model fiyatları değişiyor. Kendi kurumunuz için hesap yaparken güncel fiyat tablosunu GitHub Docs üzerinden doğrulayın.

NOTLARIM